

34. Dessiner sa propre tuile

Sortir des lettres — en chiffres, ou à la souris.

1. Ce qu'il faut comprendre

La police suffit à écrire, pas à jouer. Un **Tuile** prend **huit rangées de huit chiffres** : la nuance de chaque pixel, de 0 (le plus clair) à 3 (le plus sombre). Un **Perso** en prend seize sur seize.

Ce qui détermine le numéro d'une tuile, c'est l'ordre des déclarations. La police occupe les numéros 0 à 43 — les lettres, les chiffres, le carré plein. Le premier dessin du programme prend donc le **44**, le suivant le 45, et ainsi de suite. Le compilateur l'annonce à chaque fois, et l'atelier l'écrit sous la grille.

Le nom sert partout où un numéro est attendu : `poser(x, y, BLOC)` sur le fond, `sprite(n, x, y, BLOC)` en lutin, `lire(x, y) == BLOC` pour relire ce qui est affiché, et jusque dans une table de décor — `const uint8_t LIGNE[] = { BLOC, 0, BLOC };`. Il se calcule aussi : `BLOC + 1` est la tuile d'à côté. Un **Perso** de seize, lui, **compte pour quatre numéros** — c'est l'objet de la leçon suivante.

C'est exactement **pourquoi on écrit le nom et jamais le numéro**. Ajoute un **Perso** avant ta tuile, et elle passe de 44 à 48 : un `poser(2, 2, 44)` écrit à la main afficherait soudain autre chose, sans que rien ne le signale. Le nom, lui, reste juste — c'est le compilateur qui traduit, et c'est son travail.

Les deux façons de faire, et c'est la même chose. En code : change un `0` en `3` dans le programme, et le bloc change. À la souris : clique un pixel dans la grille ci-dessous. Le programme se réécrit tout seul sous tes yeux — c'est le même texte, écrit autrement.

Les chiffres ne sont pas la seule écriture. Huit chiffres à la file ne ressemblent pas à un dessin ; les signes `. - + #` disent exactement la même chose — le point est le vide, le dièse est le plein — et se relisent de loin : `"#.++++.#" .` Les deux donnent les mêmes octets. On n'en mélange pas deux dans une même tuile, et l'atelier rend à chacune la sienne quand tu peins.

C'est vrai partout dans ce projet : **le programme reste la seule vérité**. L'atelier ne garde aucun dessin de son côté ; il lit les `Tuile NOM = {...}` du texte, et il les réécrit. Un éditeur graphique qui tiendrait ses propres dessins finirait par ne plus dire la même chose que le code.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **411 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         52 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
AdresseCase    10 o ← poser() – trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         128 o ← CopierTuiles – 7 lettres sur 43, et 1 tuile dessinée
"MA TUILE"     8 o ← texte() – un texte gravé
```

Lettres gravées : AEILMTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

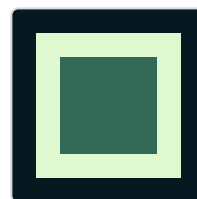
```
poser() × 1    15 o sur place, dans main() + AdresseCase (10 o)
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>     // écrit un texte à l'écran
#include <poser>     // pose une tuile sur une case du fond
```

2. UN DESSIN — BLOC

```
Tuile BLOC = {
  "33333333",
  "30000003",
  "30222203",
  "30222203",
  "30222203",
  "30222203",
  "30222203",
  "30000003",
  "33333333",
};
```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  texte(4, 2, "MA TUILE");

  for (uint8_t x = 0; x < 8; x++) {
    poser(6 + x, 8, BLOC);
  }

  while (true) {
    image();
  }

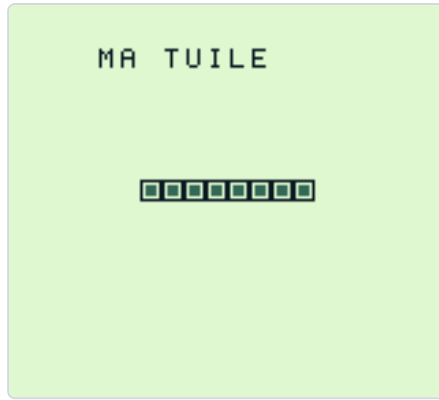
  return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une rangée de huit blocs à cadre noir, au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la tuile est posée huit fois
- ✓ elle porte le numéro 44, après la police
- ✓ et l'écran montre plus d'une nuance

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

BLOC

+ tuile 8 × 8

+ perso 16 × 16

+ perso 32 × 32

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BLOC

tuile n° 44 — poser(x, y, BLOC)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

×2

÷2

Agrandir...

Vider le dessin

Taille voulue :

✓ Créer la copie

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BLOC ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BLOC peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

Ensuite : 35. Le dessin écrit là où on le pose — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++